

Teach Once, Use Often

Building AI Skills for Data Analysis

Tyson Barrett · posit::conf(2026)

Teach On

Building AI Skills for D

Tyson Barrett · posit::conf(2026







Environmental impact

Stochastic output

Skill atrophy

Verbose code

“AI is like a box of chocolates. You never know what you're gonna get.”

Forrest Gump

Task offloading

Shallow learning



Delegate

Trust AI to do the work entirely (or errors are easy to catch).

Collaborate

AI does the typing or planning. But you own the decisions.

Own

You own the judgment, the decisions, the typing, the planning, the output.

Delegate

Trust AI to do the work entirely (or errors are easy to catch).

Slide drafts, Quarto/Jupyter/Rmarkdown scaffolding, color palettes, code documentation, data labeling.

The model specs, the estimand, the interpretation, improving code performance, reviewing code/output.

Own

You own the judgment, the decisions, the typing, the planning, the output.

Collaborate

AI does the typing or planning. But you own the decisions.

Drafting code, planning the analysis, exploratory data analysis, brainstorming solutions to data issues.

Pause and



1

CLAUDE.md (or Agents.md)

Response Style

- Be concise. Skip preamble and restating question.
- Prefer code and tables over prose.

Code Style

- Write clear code over clever code.
- Make small, reviewable changes. Explain what changed and why briefly.
- Default to R for data cleaning and wrangling and statistical models; default to python for machine learning. For all others, ask me.
- Use ``ggplot2`` and ``plotnine`` for data graphics
- Do not write comments that restate the code. Comment intent only.
- Manage python environments with ``uv`` (``uv add``, ``uv run``). Do not use bare ``pip install``.
- Descriptive and summary tables (``furniture::table1()``, ``gtsummary``)
- Regression, multilevel, and Bayesian models (``lme4``, ``brms``)
- Tables for rendering (``great_tables``); summaries via ``tableone``
- For structural equation modeling and measurement models, use ``lavaan``, ``psych``, ``semTools`` in R
- Never commit data, credentials, or ``.env`` files.
- Don't push, rebase, or force anything without asking.
- In R, use ``<-`` for assignment. Two-space indentation. ``snake_case`` names.

Data Analysis

- When doing exploratory data analysis, provide graphics over tables.
- When cleaning the data, do not change the data without asking me
- State your assumptions about all joins and filters
- Folder structure should look something like this:

```

...
data/raw/ # never modified
data/derived/
R/ or src/ # functions
analysis/ # numbered scripts or Quarto docs
output/ # tables, figures
...

```

Make Errors Visible

- Never drop NAs
- Create checks after any major data step
- Do not change the data to pass a check without asking.
- Run code to verify it works before telling me it does. If you couldn't run it, say so.
- Don't fabricate package functions, arguments, or citations. If unsure, check the docs or tell me you're unsure.
- Don't overwrite raw data.
- Don't delete files or outputs without asking.
- Don't claim results are significant, meaningful, or causal. Report the numbers; I interpret them.

Statistical Modeling

- I own every modeling decision.
- You can make recommendations but when coding, leave a `#TODO` where the models should go.
- If you think my specification is wrong, say so and explain why but do not make changes unless I ask for it.

Teaching

- For teaching materials, default to python using ``pandas``, ``statsmodels``, ``bambi``, and ``plotnine``
- Use Quarto for course materials.

2

SKILL.md

```

---
name: quarto-analysis
description: Builds reproducible Quarto (.qmd) documents for data analysis in R or Python. Use this skill whenever the user wants to create, restructure, or render a Quarto or R Markdown analysis, an analysis report, a notebook-style write-up of results, or says things like "write this up in Quarto", "make an analysis doc", "turn this script into a report", even if they don't say "Quarto" explicitly. Use when the user wants an HTML report, a PDF report, slides for a presentation, or a Word document presenting data.
---

# Quarto Data Analysis Documents

## Workflow
1. **Clarify scope.** Confirm the research question, data source, language (R or Python), and output format (HTML by default). If a project already exists, match its language and conventions.
2. **Start from the template.** Copy `assets/analysis-template-r.qmd` or `assets/analysis-template-py.qmd`. Don't write the YAML from scratch.
3. **Fill sections in order:** setup → data import → data checks → cleaning → descriptives → analysis (as specified) → diagnostics → session info.
4. **Choose chunk options deliberately.** Read `references/chunk-options.md` whenever you add figures, tables, cached chunks, or parameters.
5. **Report statistics correctly.** Read `references/stats-reporting.md` before writing any results prose, tables of estimates, or inline numbers.
6. **Render and verify** (see "Render checks"). A document that hasn't rendered isn't done.

## Document structure
Use these top-level sections. Skip ones that don't apply; don't invent new top-level ones without reason.
...

# Purpose question, estimand/target in plain language, data source
# Setup packages, options, seed, paths (chunk hidden in output)
# Data import, dimensions, codebook/variable glossary
# Data checks types, missingness, duplicates, ranges, key assumptions
# Cleaning every exclusion with counts before/after
# Descriptives Table 1, distributions, key bivariate plots
# Analysis models exactly as specified by the analyst
# Diagnostics assumption checks, convergence, sensitivity (as requested)
# Summary results with uncertainty; limitations
# Reproducibility session info, package versions, render date
...

## Core rules (and why)

- **Be brief in prose.** Do not explain things in long paragraphs. Keep any commentary short.
- **Label every chunk.** Labels make errors traceable, enable caching, and are required for cross-references. Use `fig-` and `tbl-` prefixes for figures and tables.
- **Use `#` chunk options, not `{r, echo=FALSE}`. Hash-pipe syntax works in both knitr and Jupyter engines and keeps options readable.
- **Put shared defaults in YAML `execute:`, override per chunk only when needed.
- **Never hard-code numbers in prose.** Use inline code (`{r x}` or `{python} x`) so text and results can't drift apart.
- **Keep heavy logic out of the .qmd.** Source functions from `R/` or `src/`. The document should read as a narrative with short chunks.
- **Raw data is read-only.** Read from `data/raw/`, write derived data to `data/derived/`.
- **Set a seed once in setup** and state it in the text, for anything stochastic.
- **Show exclusions as a flow.** Report n at each cleaning step; readers need to know who the results apply to.
- **Don't suppress warnings globally for model chunks.** Convergence and singular-fit warnings are results. Suppress package-load messages only.
- **Missing data is reported, never silently dropped.** State the n used in every table and model.

## Language defaults

**R (knitr engine):** `tidyverse` for wrangling, `furniture::table1()` or `gtsummary` for descriptive tables, `ggplot2` for figures, `broom`/`broom.mixed` or `parameters` for tidying estimates, `gt` or `great_tables`-style output for display. Use `renv` if the project has a lockfile.

**Python (jupyter engine):** `polars` or `pandas`, `tableone` for descriptives, `great_tables` for display, `statsmodels`/`pingouin` for classical stats, `plotnine` for figures (unless built in function is easy and clear). Environments via `uv`; render with `uv run quarto render`.

## Render checks
After rendering, confirm:
- [ ] Render completes from a clean session (`quarto render analysis.qmd`) with no errors
- [ ] No unexpected warnings in the output (expected ones are explained in text)
- [ ] Every figure and table has a caption
- [ ] Every number in prose comes from inline code
- [ ] n is stated for each table and model
- [ ] Session info section is present
- [ ] If `cache: true` was used anywhere, re-render with `--cache-refresh` once before sharing

Report any check that fails instead of declaring the document finished.

## Files in this skill

- `references/chunk-options.md` — chunk and YAML options with when to use each
- `references/stats-reporting.md` — how to present estimates, uncertainty, diagnostics
- `assets/analysis-template-r.qmd` — starting template (R/knitr)
- `assets/analysis-template-py.qmd` — starting template (Python/Jupyter)

```

1

CLAUDE.md

2

SKILL.md

Response Style

- Be concise. Skip preamble and restating question.
- Prefer code and tables over prose.

Code Style

- Write clear code over clever code.
- Make small, reviewable changes. Explain changes as you go.
- Default to R for data cleaning and wrangling.
- Use 'ggplot2' and 'plotnine' for data graphics.
- Do not write comments that restate the code. Comment instead on why you are doing something.
- Manage python environments with 'uv' ('uv add', 'uv run').
- Descriptive and summary tables ('furniture::table1()', 'summary()').
- Regression, multilevel, and Bayesian models ('lme4', 'brms').
- Tables for rendering ('great_tables'); summaries via 'tableone'.
- For structural equation modeling and measurement models, use 'lavaan', 'psych', 'semTools' in R.
- Never commit data, credentials, or '.env' files.
- Don't push, rebase, or force anything without asking.
- In R, use '<-' for assignment. Two-space indentation. 'snake_case' names.

Data Analysis

- When doing exploratory data analysis, provide graphics over tables.
- When cleaning the data, do not change the data without asking me.
- State your assumptions about all joins and filters.
- Folder structure should look something like this:

```
...
data/raw/ # never modified
data/derived/
R/ or src/ # functions
analysis/ # numbered scripts or Quarto docs
output/ # tables, figures
```

Make Errors Visible

- Never drop NAs.
- Create checks after any major data step.
- Do not change the data to pass a check without asking.
- Run code to verify it works before telling me it does. If you couldn't run it, say so.
- Don't fabricate package functions, arguments, or citations. If unsure, check the docs or tell me you're unsure.
- Don't overwrite raw data.
- Don't delete files or outputs without asking.
- Don't claim results are significant, meaningful, or causal. Report the numbers; I interpret them.

Statistical Modeling

- I own every modeling decision.
- You can make recommendations but when coding, leave a #TODO where the models should go.
- If you think my specification is wrong, say so and explain why but do not make changes unless I ask for it.

Teaching

- For teaching materials, default to python using 'pandas', 'statsmodels', 'bambi', and 'plotnine'.
- Use Quarto for course materials.

Response Style

- Be concise. Skip preamble and restating question.
- Prefer code and tables over prose.

1

CLAUDE.md

2

SKILL.md

Response Style

- Be concise. Skip preamble and restating question.
- Prefer code and tables over prose.

Code Style

- Write clear code over clever code.
- Make small, reviewable changes. Explain what changed and why briefly.
- Default to R for data cleaning and wrangling and statistical models; default to python for machine learning. For all others, ask me.
- Use ``ggplot2`` and ``plotnine`` for data graphics
- Do not write comments that restate the code. Comment intent only.
- Manage python environments with ``uv`` (``uv add``, ``uv run``). Do not use bare ``pip install``.
- Descriptive and summary tables (``furniture::table1()``, ``gtsummary``)
- Regression, multilevel, and Bayesian models (``lme4``, ``brms``)
- Tables for rendering (``great_tables``); summaries via ``tableone``
- For structural equation modeling and measurement models, use ``lavaan``, ``psych``, ``sem``
- Never commit data, credentials, or ``.env`` files.
- Don't push, rebase, or force anything without asking.
- In R, use ``<-`` for assignment. Two-space indentation. ``snake_case`` names.

Data Analysis

- When doing exploratory data analysis, provide graphics over tables
- When cleaning the data, do not change the data without asking me
- State your assumptions about all joins and filters
- Folder structure should look something like this:

```
...
data/raw/ # never modified
data/derived/
R/ or src/ # functions
analysis/ # numbered scripts or Quarto docs
output/ # tables, figures
```

Make Errors Visible

- Never drop NAs
- Create checks after any major data step
- Do not change the data to pass a check without asking.
- Run code to verify it works before telling me it does. If you couldn't run it, say so.
- Don't fabricate package functions, arguments, or citations. If unsure, check the docs.
- Don't overwrite raw data.
- Don't delete files or outputs without asking.
- Don't claim results are significant, meaningful, or causal. Report the numbers; I'll interpret.

Statistical Modeling

- I own every modeling decision.
- You can make recommendations but when coding, leave a `#TODO` where the models should go.
- If you think my specification is wrong, say so and explain why but do not make changes unless I ask for it.

Teaching

- For teaching materials, default to python using ``pandas``, ``statsmodels``, ``bambi``, and ``plotnine``
- Use Quarto for course materials.

Code Style

- Use ``ggplot2`` and ``plotnine`` for data graphics
- Do not write comments that restate the code. Comment intent only.
- Use functions from packages rather than writing new complex functions.
- Descriptive and summary tables (``furniture::table1()``, ``gtsummary``)

1

CLAUDE.md

2

SKILL.md

```
# Response Style
- Be concise. Skip preamble and restating question.
- Prefer code and tables over prose.

# Code Style
- Write clear code over clever code.
- Make small, reviewable changes. Explain what changed and why briefly.
- Default to R for data cleaning and wrangling and statistical models; default to python for machine learning. For all others, ask me.
- Use 'ggplot2' and 'plotnine' for data graphics
- Do not write comments that restate the code. Comment intent only.
- Manage python environments with 'uv' ('uv add', 'uv run'). Do not use bare 'pip install'.
- Descriptive and summary tables ('furniture::table1()', 'gtsummary')
- Regression, multilevel, and Bayesian models ('lme4', 'brms')
- Tables for rendering ('great_tables'); summaries via 'tableone'
- For structural equation modeling and measurement models, use 'lavaan', 'psych',
- Never commit data, credentials, or '.env' files.
- Don't push, rebase, or force anything without asking.
- In R, use '<-' for assignment. Two-space indentation. 'snake_case' names.

# Data Analysis
- When doing exploratory data analysis, provide graphics over tables.
- When cleaning the data, do not change the data without asking me
- State your assumptions about all joins and filters
- Folder structure should look something like this:
...
data/raw/ # never modified
data/derived/
R/ or src/ # functions
analysis/ # numbered scripts or Quarto docs
output/ # tables, figures

# Make Errors Visible
- Never drop NAs
- Create checks after any major data step
- Do not change the data to pass a check without asking.
- Run code to verify it works before telling me it does. If you couldn't run it, say so.
- Don't fabricate package functions, arguments, or citations. If unsure, check the docs.
- Don't overwrite raw data.
- Don't delete files or outputs without asking.
- Don't claim results are significant, meaningful, or causal. Report the numbers; I will interpret.

# Statistical Modeling
- I own every modeling decision.
- You can make recommendations but when coding, leave a #TODO where the models should go.
- If you think my specification is wrong, say so and explain why but do not make changes unless I ask for it.

# Teaching
- For teaching materials, default to python using 'pandas', 'statsmodels', 'bambi', and 'plotnine'
- Use Quarto for course materials.
```



Make Errors Visible

- Never drop NAs
- Create checks after any major data step
- Do not change the data to pass a check without asking.
- Don't overwrite raw data.
- Don't delete files or outputs without asking.
- When updating a file, create a backup so no other changes get lost.

1

CLAUDE.md

```
# Response Style
- Be concise. Skip preamble and restating question.
- Prefer code and tables over prose.

# Code Style
- Write clear code over clever code.
- Make small, reviewable changes. Explain what changed and why briefly.
- Default to R for data cleaning and wrangling and statistical models; default to python for machine learning. For all others, ask me.
- Use 'ggplot2' and 'plotnine' for data graphics
- Do not write comments that restate the code. Comment intent only.
- Manage python environments with 'uv' ('uv add', 'uv run'). Do not use bare 'pip install'.
- Descriptive and summary tables ('furniture::table1()', 'gtsummary')
- Regression, multilevel, and Bayesian models ('lme4', 'brms')
- Tables for rendering ('great_tables'); summaries via 'tableone'
- For structural equation modeling and measurement models, use 'lavaan', 'psych', 'semTools' in R
- Never commit data, credentials, or '.env' files.
- Don't push, rebase, or force anything without asking.
- In R, use '<->' for assignment. Two-space indentation. 'snake_case' names.

# Data Analysis
- When doing exploratory data analysis, provide graphics over tables.
- When cleaning the data, do not change the data without asking me
- State your assumptions about all joins and filters
- Folder structure should look something like this:
...
data/raw/ # never modified
data/derived/
R/ or src/ # functions
analysis/ # numbered scripts or Quarto docs
output/ # tables, figures

# Make Errors Visible
- Never drop NAs
- Create checks after any major data step
- Do not change the data to pass a check without asking.
- Run code to verify it works before telling me it does. If you couldn't tell, say so.
- Don't fabricate package functions, arguments, or citations. If unsure, check the docs.
- Don't overwrite raw data.
- Don't delete files or outputs without asking.
- Don't claim results are significant, meaningful, or causal. Report the numbers; I'll interpret.

# Statistical Modeling
- I own every modeling decision.
- You can make recommendations but when coding, leave a #TODO where the models should go.
- If you think my specification is wrong, say so and explain why but do not make changes unless I ask for it.

# Teaching
- For teaching materials, default to python using 'pandas', 'statsmodels', 'bambi', and 'plotnine'
- Use Quarto for course materials.
```

2

SKILL.md

Statistical Modeling

- I own every modeling decision.
- You can make recommendations but when coding, leave a #TODO where the models should go.
- If you think my specification is wrong, say so and explain why but do not make changes unless I ask for it.

1

CLAUDE.md

2

SKILL.md

name: quarto-analysis

description: Builds reproducible Quarto (.qmd) documents for data analysis in R or Python. Use this skill whenever the user wants to create, restructure, or render a Quarto or R Markdown analysis, an analysis report, a notebook-style write-up of results, or says things like "write this up in Quarto", "make an analysis doc", "turn this script into a report", even if they don't say "Quarto" explicitly. Use when the user wants an HTML report, a PDF report, slides for a presentation, or a Word document presenting data.

```
---
name: quarto-analysis
description: Builds reproducible Quarto (.qmd) documents for data analysis in R or Python. Use this skill whenever the user wants to create, restructure, or render a Quarto or R Markdown analysis, an analysis report, a notebook-style write-up of results, or says things like "write this up in Quarto", "make an analysis doc", "turn this script into a report", even if they don't say "Quarto" explicitly. Use when the user wants an HTML report, a PDF report, slides for a presentation, or a Word document presenting data.
---
```

(R or Python) and output format (HTML by default). If a project already exists, match its assets/analysis-template-py.qmd. Don't write the YAML from scratch. eg → diagnostics → analysis (as specified) → diagnostics → session info. Whenever you add figures, tables, cached chunks, or parameters. Before writing any results prose, tables of estimates, or inline numbers. Consider isn't done.

Now top level ones

by commentary short. are required for cross-references. Use 'fig-' and 'tbl-' prefixes for figures and tables. as in both knitr and Jupyter engines and keeps options readable. n needed. (python) x'`) so text and results can't drift apart. z/'. The document should read as a narrative with short chunks. 'data/derived/'. chastic. need to know who the results apply to. singular-fit warnings are results. Suppress package-load messages only. in every table and model.

'gtsummary' for descriptive tables, 'ggplot2' for figures, 'broom'/'broom.mixed' or or display. Use 'renv' if the project has a lockfile.

atives, 'great_tables' for display, 'statsmodels'/'pingouin' for classical stats, 'plotnine' 'uv'; render with

with no errors text)

- [] If 'cache: true' was used anywhere, re-render with '--cache-refresh' once before sharing

Report any check that fails instead of declaring the document finished.

Files in this skill

- 'references/chunk-options.md' - chunk and YAML options with when to use each
- 'references/stats-reporting.md' - how to present estimates, uncertainty, diagnostics
- 'assets/analysis-template-r.qmd' - starting template (R/knitr)
- 'assets/analysis-template-py.qmd' - starting template (Python/Jupyter)

1

CLAUDE.md

2

SKILL.md

```

---
name: quarto-analysis
description: Builds reproducible Quarto (.qmd) documents for data analysis in R or Python. Use this skill whenever the user wants to create, restructure, or render a Quarto or R Markdown analysis, an analysis report, a notebook-style write-up of results, or says things like "write this up in Quarto", "make an analysis doc", "turn this script into a report", even if they don't say "Quarto" explicitly. Use when the user wants an HTML report, a PDF report, slides for a presentation, or a Word document presenting data.
---

# Quarto Data Analysis Documents

## Workflow
1. Clarify scope. Confirm the research question, data source, language (R or Python), and output format (HTML by default). If a project already exists, match its language and conventions.
2. Start from the template. Copy `assets/analysis-template-r.qmd` or `assets/analysis-template-py.qmd`. Don't write the YAML from scratch.
3. Fill sections in order: setup → data import → data checks → cleaning → descriptives → analysis (as specified) → diagnostics → session info.
4. Choose chunk options deliberately. Read `references/chunk-options.md` whenever you add figures, tables, cached chunks, or parameters.
5. Report statistics correctly. Read `references/stats-reporting.md` before writing any results prose, tables of estimates, or inline numbers.
  
```

Workflow

1. **Clarify scope.** Confirm the research question, data source, language (R or Python), and output format (HTML by default). If a project already exists, match its language and conventions.

2. **Start from the template.** Copy `assets/analysis-template-r.qmd` or `assets/analysis-template-py.qmd`. Don't write the YAML from scratch.

3. **Fill sections in order:** setup → data import → data checks → cleaning → descriptives → analysis (as specified) → diagnostics → session info.

...



1

CLAUDE.md

2

SKILL.md

```

---
name: quarto-analysis
description: Builds reproducible Quarto (.qmd) documents for data analysis in R or Python. Use this skill whenever the user wants to create, restructure, or render a Quarto or R Markdown analysis, an analysis report, a notebook-style write-up of results, or says things like "write this up in Quarto", "make an analysis doc", "turn this script into a report", even if they don't say "Quarto" explicitly. Use when the user wants an HTML report, a PDF report, slides for a presentation, or a Word document presenting data.
---

# Quarto Data Analysis Documents

## Workflow
1. **Clarify scope.** Confirm the research question, data source, language (R or Python), and output format (HTML by default). If a project already exists, match its language and conventions.
2. **Start from the template.** Copy `assets/analysis-template-r.qmd` or `assets/analysis-template-py.qmd`. Don't write the YAML from scratch.
3. **Fill sections in order**: setup → data import → data checks → cleaning → descriptives → analysis (as specified) → diagnostics → session info.
4. **Choose chunk options deliberately.** Read `references/chunk-options.md` whenever you add figures, tables, cached chunks, or parameters.
5. **Report statistics correctly.** Read `references/stats-reporting.md` before writing any results prose, tables of estimates, or inline numbers.
  
```

Workflow

1. ****Clarify scope.**** Confirm the research question, data source, language (R or Python), and output format (HTML by default). If a project already exists, match its language and conventions.

2. ****Start from the template.**** Copy `assets/analysis-template-r.qmd` or `assets/analysis-template-py.qmd``. Don't write the YAML from scratch.

3. ****Fill sections in order**:** setup → data import → data checks → cleaning → descriptives → analysis (as specified) → diagnostics → session info.

...



1

CLAUDE.md

2

SKILL.md

- ****Be brief in prose.**** Keep any commentary short.
- ****Label every chunk.**** Use ``fig-`` and ``tbl-`` prefixes for figures and tables.
- ****Use ``#|`` chunk options, not ``{r, echo=FALSE}``.** Hash-pipe syntax works in both knitr and Jupyter engines and keeps options readable.
- ****Never hard-code numbers in prose.**** Use inline code (`` `r x` `` or `` `python x` ``) so text and results can't drift apart.
- ****Keep heavy logic out of the .qmd.**** Source functions from ``R/`` or ``src/``. The document should read as a narrative with short chunks.
- ****Raw data is read-only.**** Read from ``data/raw/``, write derived data to ``data/derived/``.
- ****Report n at each cleaning step**.**
- ****Don't suppress warnings globally for model chunks.**** Convergence and singular-fit warnings are results. Suppress package-load messages only.

```
name: quarto-analysis
description: Builds reproducible Quarto (.qmd) documents for data analysis in R or Python. Use this skill whenever the user wants to create, restructure, or render a
  rt, a notebook-style write-up of results, or says things like "write this up in Quarto", "make an analysis doc", "turn
  say "Quarto" explicitly. Use when the user wants an HTML report, a PDF report, slides for a presentation, or a Word
  doc.

  Clean, data source, language (R or Python), and output format (HTML by default). If a project already exists, match its
  style/template-r.qmd or assets/analysis-template-py.qmd. Don't write the YAML from scratch.
  sort -> data checks -> cleaning -> descriptives -> analysis (as specified) -> diagnostics -> session info.
  references/chunk-options.qmd whenever you add figures, tables, cached chunks, or parameters.
  notes/stats-reporting.qmd before writing any results prose, tables of estimates, or inline numbers.
  A document that hasn't rendered isn't done.

  Don't apply; don't invent new top-level ones.

  language, data source
  (shown in output)
  session
  session
  state, plots
  output
  modality (as HTML, PDF)
  render date

  in long paragraphs. Keep any commentary short.
  eable, enable caching, and are required for cross-references. Use 'fig-' and 'tbl-' prefixes for figures and tables.
  }".** Hash-pipe syntax works in both knitr and Jupyter engines and keeps options readable.
  override per chunk only when needed.
  line code ("`r x`" or "{python} x`") so text and results can't drift apart.
  functions from 'R/' or 'src/'. The document should read as a narrative with short chunks.
  w/, write derived data to 'data/derived/'.
  the text, for anything stochastic.
  ch cleaning step; readers need to know who the results apply to.
  chunks.** Convergence and singular-fit warnings are results. Suppress package-load messages only.
  topped.** State the n used in every table and model.

  . 'furniture::tbl_md()' or 'summary' for descriptive tables, 'ggplot2' for figures, 'brake/' 'brake.qmd' or
  net-tables-style output for display. Use 'new' if the project has a lockfile.

  . 'tableone' for descriptives, 'great_tables' for display, 'statsmodels/' 'pinguin' for classical stats, 'plotnine
  and clear). Environments via 'w/'; render with

  Quarto render analysis.qmd with no errors
  ected ones are explained in text)

  ide

  render with '--cache-refresh' once before sharing
  e the document finished.

  options with when to use each
  ent estimates, uncertainty, diagnostics
  eplace (R/knitr)
  template (python/Jupyter)
```

1

CLAUDE.md

Response Style

- Be concise. Skip preamble and restating question.
- Prefer code and tables over prose.

DELEGATE

Code Style

- Write clear code over clever code.
- Make small, reviewable changes. Explain what changed and why briefly.
- Default to R for data cleaning and wrangling and statistical models; default to python for machine learning. For all others, ask me.
- Use `'ggplot2'` and `'plotnine'` for data graphics
- Do not write comments that restate the code. Comment intent only.
- Manage python environments with `'uv'` (`'uv add'`, `'uv run'`). Do not use bare `'pip install'`.
- Descriptive and summary tables (`'furniture::table1()'`, `'gtsummary'`)
- Regression, multilevel, and Bayesian models (`'lme4'`, `'brms'`)
- Tables for rendering (`'great_tables'`); summaries via `'tableone'`
- For structural equation modeling and measurement models, use `'lavaan'`, `'psych'`, `'semTools'` in R
- Never commit data, credentials, or `'.env'` files.
- Don't push, rebase, or force anything without asking.
- In R, use `'<-'` for assignment. Two-space indentation. `'snake_case'` names.

DELEGATE

Data Analysis

- When doing exploratory data analysis, provide graphics over tables
- When cleaning the data, do not change the data without asking me
- State your assumptions about all joins and filters
- Folder structure should look something like this:

COLLABORATE

```
...
data/raw/ # never modified
data/derived/
R/ or src/ # functions
analysis/ # numbered scripts or Quarto docs
output/ # tables, figures
...
```

Make Errors Visible

- Never drop NAs
- Create checks after any major data step
- Do not change the data to pass a check without asking me
- Run code to verify it works before telling me it does. If you couldn't run it, say so.
- Don't fabricate package functions, arguments, or citations. If unsure, check the docs or tell me you're unsure.
- Don't overwrite raw data.
- Don't delete files or outputs without asking.
- Don't claim results are significant, meaningful, or causal. Report the numbers; I interpret them.

DELEGATE

Statistical Modeling

OWN

- I own every modeling decision.
- You can make recommendations but when coding, leave a `#TODO` where the models should go.
- If you think my specification is wrong, say so and explain why but do not make changes unless I ask for it.

Teaching

- For teaching materials, default to python using `'pandas'`, `'statsmodels'`, `'bambi'`, and `'plotnine'`
- Use Quarto for course materials.

COLLABORATE

2

SKILL.md

```
---
name: quarto-analysis
description: Builds reproducible Quarto (.qmd) documents for data analysis in R or Python. Use this skill whenever the user wants to create, restructure, or render a Quarto or R Markdown analysis, an analysis report, a notebook-style write-up of results, or says things like "write this up in Quarto", "make an analysis doc", "turn this script into a report", even if they don't say "Quarto" explicitly. Use when the user wants an HTML report, a PDF report, slides for a presentation, or a Word document presenting data.
---
```

Quarto Data Analysis Documents

Workflow

1. **Clarify scope.** Confirm the research question, data source, language (R or Python), and output format (HTML by default). If a project already exists, match its language and conventions.
2. **Start from the template.** Copy `'assets/analysis-template-r.qmd'` or `'assets/analysis-template-py.qmd'`. Don't write the YAML from scratch.
3. **Fill sections in order:** setup → data import → data checks → cleaning → descriptives → analysis (as specified) → diagnostics → session info.
4. **Choose chunk options deliberately.** Read `'references/chunk-options.md'` whenever you add figures, tables, cached chunks, or parameters.
5. **Report statistics correctly.** Read `'references/stats-reporting.md'` before writing any results prose, tables of estimates, or inline numbers.
6. **Render and verify** (see "Render checks"). A document that hasn't rendered isn't done.

COLLABORATE

Document structure

Use these top-level sections. Skip ones that don't apply; don't invent new top-level ones without reason.

```
...
# Purpose question, estimand/target in plain language, data source
# Setup packages, options, seed, paths (chunk hidden in output)
# Data import, dimensions, codebook/variable glossary
# Data checks types, missingness, duplicates, ranges, key assumptions
# Cleaning every exclusion with counts before/after
# Descriptives Table 1, distributions, key bivariate plots
# Analysis models exactly as specified by the analyst
# Diagnostics assumption checks, convergence, sensitivity (as requested)
# Summary results with uncertainty; limitations
# Reproducibility session info, package versions, render date
...
```

DELEGATE

Core rules (and why)

- **Be brief in prose.** Do not explain things in long paragraphs. Keep any commentary short.
- **Label every chunk.** Labels make errors traceable, enable caching, and are required for cross-references. Use `'fig-'` and `'tbl-'` prefixes for figures and tables.
- **Use `'#|'` chunk options,** not `'(r, echo=FALSE)'`. Hash-pipe syntax works in both knitr and Jupyter engines and keeps options readable.
- **Put shared defaults in YAML `'execute:'`**, override per chunk only when needed.
- **Never hard-code numbers in prose.** Use inline code (`'(r x' or '(python) x'`) so text and results can't drift apart.
- **Keep heavy logic out of the .qmd.** Source functions from `'R/'` or `'src/'`. The document should read as a narrative with short chunks.
- **Raw data is read-only.** Read from `'data/raw/'`, write derived data to `'data/derived/'`.
- **Set a seed once in setup** and state it in the text, for anything stochastic.
- **Show exclusions as a flow.** Report `n` at each cleaning step; readers need to know who the results apply to.
- **Don't suppress warnings globally for model chunks.** Convergence and singular-fit warnings are results. Suppress package-load messages only.
- **Missing data is reported, never silently dropped.** State the `n` used in every table and model.

OWN

Language defaults

R (knitr engine): `'tidyverse'` for wrangling, `'furniture::table1()' or 'gtsummary'` for descriptive tables, `'ggplot2'` for figures, `'broom'/'broom.mixed'` or `'parameters'` for tidying estimates, `'gt'` or `'great_tables'`-style output for display. Use `'renv'` if the project has a lockfile.

Python (jupyter engine): `'polars'` or `'pandas'`, `'tableone'` for descriptives, `'great_tables'` for display, `'statsmodels'/'pingouin'` for classical stats, `'plotnine'` for figures (unless built in function is easy and clear). Environments via `'uv'`; render with `'uv run quarto render'`.

Render checks

After rendering, confirm:

- [] Render completes from a clean session (`'quarto render analysis.qmd'`) with no errors
- [] No unexpected warnings in the output (expected ones are explained in text)
- [] Every figure and table has a caption
- [] Every number in prose comes from inline code
- [] `n` is stated for each table and model
- [] Session info section is present
- [] If `'cache: true'` was used anywhere, re-render with `'--cache-refresh'` once before sharing

COLLABORATE

Report any check that fails instead of declaring the document finished.

Files in this skill

- `'references/chunk-options.md'` — chunk and YAML options with when to use each
- `'references/stats-reporting.md'` — how to present estimates, uncertainty, diagnostics
- `'assets/analysis-template-r.qmd'` — starting template (R/knitr)
- `'assets/analysis-template-py.qmd'` — starting template (Python/Jupyter)

DELEGATE

assistant.posit.co

A note about Posit Assistant



Thanks!



tyson.barrett@usu.edu



github.com/tysonstanley



tysonbarrett.com

RESOURCES



Templates and Examples

opensource.posit.co/software/skills

github.com/nimrodfisher/data-analytics-skills

opensource.posit.co/software/skills



Posit's skill library

github.com/posit-dev/skills

opensource.posit.co/software/skills



Posit AI

posit.co/blog/introducing-ai-in-rstudio